TP3: Implementação de mecanismos de QoS usando VNFs

Proposta (Fase 1) · Qualidade de Serviço na Internet · Universidade do Minho · 2026/2027 – 1.º Semestre

Grupo: [Nome 1 – nº] · [Nome 2 – nº] · [Nome 3 – nº]

1. Introdução e objetivos

Este projeto pretende implementar, de forma totalmente virtualizada, uma cadeia de Funções de Rede Virtualizadas (VNFs) que aplica mecanismos de Qualidade de Serviço (QoS) a três tipos de tráfego com requisitos distintos: VoIP (baixa latência e jitter), streaming de vídeo (elevada largura de banda) e dados em massa (best-effort). O objetivo é medir quantitativamente o impacto de cada VNF na latência, no jitter, no débito e na perda de pacotes, e discutir as contrapartidas face a abordagens baseadas em hardware dedicado.

Seguindo o modelo de Serviços Diferenciados (DiffServ) [1], [2], o grupo implementará **cinco aspetos de QoS**: classificação, controlo de acessos, policiamento, agendamento e monitorização. Os três obrigatórios (classificação, policiamento e monitorização) constituem o núcleo do projeto; o controlo de acessos e o agendamento são incluídos como extensão (bónus), sendo o agendamento essencial para que a prioridade dada ao VoIP tenha efeito real no gargalo.

2. Ambiente e ferramentas

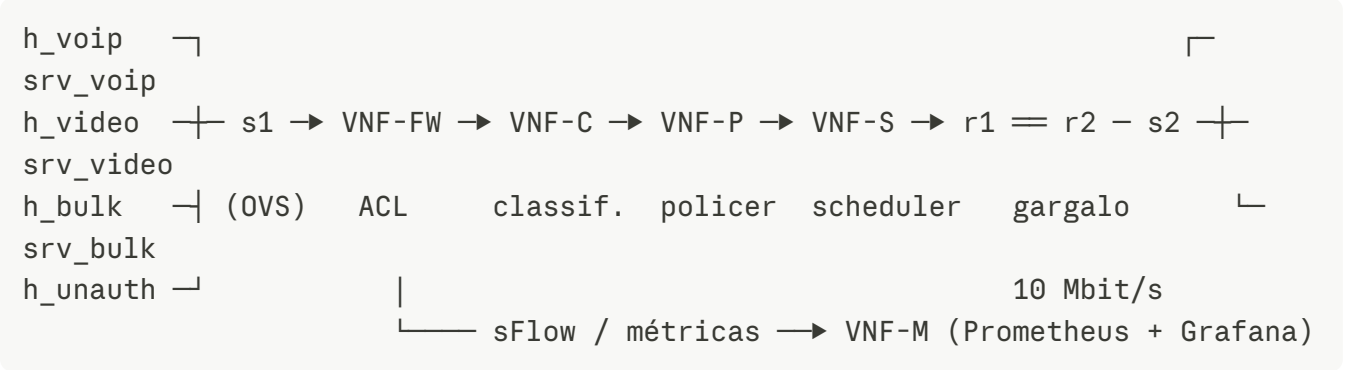
O desenvolvimento será feito em MacBook com Apple Silicon (arquitetura ARM64). Como o Mininet depende do kernel Linux, será usada uma máquina virtual Ubuntu Server 24.04 (aarch64) executada em UTM, na qual correm todas as ferramentas. Todas as ferramentas são gratuitas/open source e têm suporte ARM64, e não é necessário qualquer hardware físico.

Função	Ferramenta
Emulação de rede e VNFs em contentores	Mininet / Containernet [3] + Docker
Comutação virtual	Open vSwitch
Classificação e marcação DSCP	Python + Scapy / nftables
Controlo de acessos	nftables (firewall em contentor)

Função	Ferramenta
Policiamento e agendamento	Linux `tc` (tbf, HTB, PRIO, netem)
Monitorização	sFlow + Prometheus + Grafana
Geração de tráfego	D-ITG (VoIP), iperf3 (vídeo UDP e bulk TCP), Scapy
Orquestração/encadeamento	Scripts Python (Containernet) + Docker Compose
Controlo de versões	Git (GitHub/GitLab)

3. Topologia e arquitetura

A topologia emula o acesso a um ISP: uma rede de clientes, uma cadeia de VNFs na fronteira, um único link de gargalo (onde ocorre o congestionamento) e uma rede de servidores.



O link r1–r2 será limitado a 10 Mbit/s com 20 ms de atraso (via `tc`), garantindo que o tráfego agregado possa exceder a capacidade. Cada VNF é um contentor Docker com duas interfaces, ligado em série. A ordem da cadeia justifica-se assim: o firewall descarta primeiro o tráfego não autorizado para não gastar recursos nas VNFs seguintes; a classificação marca o DSCP; o policiador limita cada classe; e o agendador serve as filas no link de gargalo.

4. Cadeia de VNFs

VNF	Aspeto de QoS	Implementação	Comportamento previsto
VNF-FW	Controlo de acessos	nftables em contentor	Permite apenas os serviços definidos; bloqueia `h_unauth` e portas não autorizadas

VNF	Aspeto de QoS	Implementação	Comportamento previsto
VNF-C	Classificação	Scapy / nftables	Marca por IP, porta e protocolo: VoIP/RTP → EF (46); vídeo → AF41 (34); restante → BE (0) [4]
VNF-P	Policiamento	`tc` tbf / token bucket em Python	Limita a taxa de cada classe (p. ex. vídeo e bulk), descartando o excesso
VNF-S	Agendamento	`tc` PRIO + HTB	EF com prioridade estrita; AF41 e BE com partilha ponderada
VNF-M	Monitorização	sFlow, Prometheus, Grafana	Recolhe débito, perdas e ocupação por classe; painel em tempo real

5. Cenários de tráfego e métricas

Tráfego	Gerador	Perfil	Classe
VoIP	D-ITG	UDP, G.711 (64 kbit/s, 50 pkt/s, payload 160 B)	EF
Vídeo	iperf3 UDP	Débito constante ~4 Mbit/s	AF41
Dados em massa	iperf3 TCP	Fluxo contínuo (tenta saturar o link)	BE
Não autorizado	Scapy/iperf3	Origem/portas bloqueadas	–

Métricas: atraso (RTT e unidireccional), jitter, débito, perda de pacotes e, para VoIP, uma estimativa de MOS (modelo E).

Plano de experiências: (i) baseline sem QoS; (ii) cada VNF isolada; (iii) cadeia completa; (iv) variação da carga (50% a 150% do gargalo); (v) condições adversas com `tc netem` (atraso, perda, variação de carga). Cada cenário é repetido várias vezes, reportando médias e desvio padrão, e as medições são comparadas antes/depois da aplicação de cada VNF.

6. Cronograma

Tarefa	S1	S2	S3	S4	S5	S6	S7
Setup da VM, Containernet e topologia	■	■					
Desenho da arquitetura e cenários de tráfego	■	■					

Tarefa	S1	S2	S3	S4	S5	S6	S7
Entrega: proposta		■	■				
VNF-FW e VNF-C			■				
VNF-P e VNF-S			■	■			
VNF-M e encadeamento				■			
Testes isolados das VNFs			■	■			
Integração da cadeia completa					■		
Entrega: relatório de progresso					■		
Medições antes/depois e condições adversas					■	■	
Otimização de parâmetros						■	
Experiências finais e análise							■
Relatório final, vídeo e repositório							■

Riscos e mitigação: (1) instalação do Containernet em ARM64: alternativa com Mininet puro e namespaces/Docker; (2) tempo para os cinco aspetos: garantir primeiro o núcleo (classificação, policiamento, monitorização) e só depois FW e agendador; (3) variabilidade das medições na VM: repetir experiências e fixar a carga da máquina.

Referências

[1] K. Nichols, S. Blake, F. Baker e D. Black, “Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers,” RFC 2474, IETF, 1998.

[2] S. Blake et al., “An Architecture for Differentiated Services,” RFC 2475, IETF, 1998.

[3] B. Lantz, B. Heller e N. McKeown, “A network in a laptop: rapid prototyping for software-defined networks,” in *Proc. ACM HotNets-IX*, 2010.

[4] J. Babiarz, K. Chan e F. Baker, “Configuration Guidelines for DiffServ Service Classes,” RFC 4594, IETF, 2006.

[5] ETSI, “Network Functions Virtualisation (NFV); Architectural Framework,” ETSI GS NFV 002.